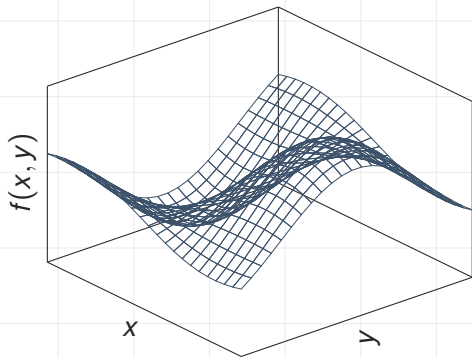


Ecuaciones No Lineales

Métodos Numéricos para la Localización
de Raíces

Autor: Dr. Hermes Pantoja Carhuavilca

Curso: Métodos Numéricos

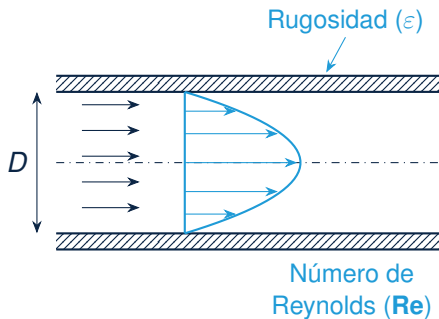


El Desafío: Fricción en Tuberías

En ingeniería de fluidos, determinar la caída de presión en una tubería es crucial. Esto requiere calcular el **factor de fricción** f , que se rige por la ecuación implícita de **Colebrook** para flujo turbulento:

$$\frac{1}{\sqrt{f}} = -2.0 \log_{10} \left(\frac{\varepsilon/D}{3.7} + \frac{2.51}{\text{Re}\sqrt{f}} \right)$$

Punto clave: esta ecuación es **no lineal** e **implícita**: la incógnita f aparece en ambos lados y no puede despejarse con álgebra simple. *¿Cómo resolvemos este problema fundamental?*

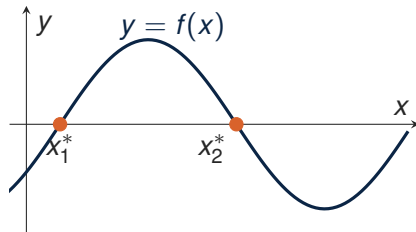


Fundamentos: ¿Qué Buscamos Exactamente?

Concepto: buscamos la **raíz** (o **cero**) de una función. Una raíz x^* es un valor de la variable independiente x para el cual

$$f(x^*) = 0.$$

Interpretación gráfica: las raíces son los puntos donde la gráfica de $y = f(x)$ **crusa el eje X**.



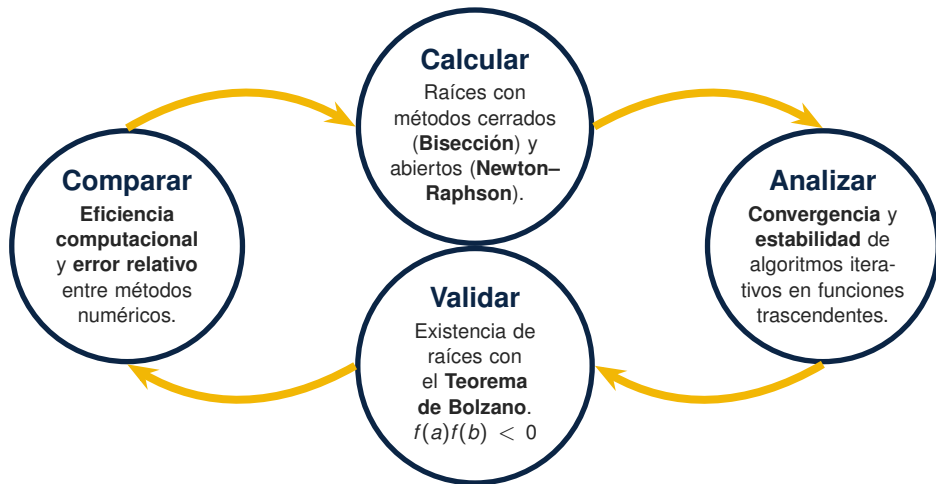
Reformulación del problema

El problema de fricción se reescribe como la búsqueda de la raíz de:

$$F(f) = \frac{1}{\sqrt{f}} + 2.0 \log_{10} \left(\frac{\varepsilon/D}{3.7} + \frac{2.51}{\text{Re}\sqrt{f}} \right) = 0$$

Objetivos de la Sesión

Al finalizar la sesión, el estudiante habrá adquirido las competencias para:



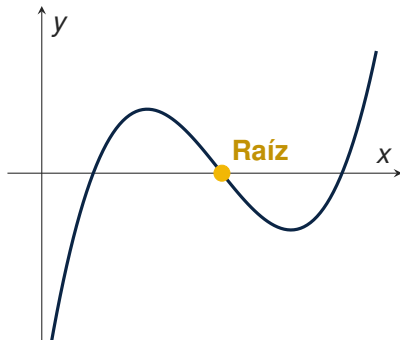
Definición del Problema

El objetivo fundamental es hallar un valor x tal que se cumpla la **condición de nulidad**:

$$f(x) = 0$$

A diferencia de las ecuaciones lineales, las funciones **trascendentes** o **polinómicas de alto grado** no siempre permiten un despeje analítico directo.

La solución: métodos **iterativos** que refinan una aproximación inicial hasta que el error relativo aproximado $|\varepsilon_a|$ sea menor que una tolerancia predefinida.



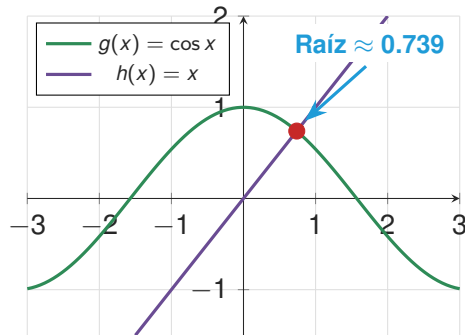
A. Localización de Raíces: Método Gráfico

El primer paso: inspección visual

- **Técnica:** graficar $f(x)$ o separar en $g(x) = h(x)$ y buscar la intersección.
- **Ventajas:** ✓ intuición visual para seleccionar x_0 o el intervalo $[a, b]$.
- **Desventajas:** ✗ impreciso, depende de la escala (*zoom*).

Ejemplo:

$$f(x) = x - \cos x = 0 \iff x = \cos x$$



Herramienta 1: El Método Gráfico

La inspección visual

Concepto: graficar $y = f(x)$ y observar dónde cruza el eje X .

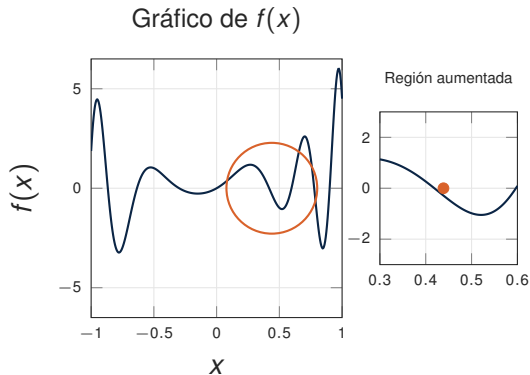
Proceso:

1. Generar un conjunto de puntos $(x, f(x))$ en un intervalo de interés.
2. Graficar estos puntos.
3. Identificar la(s) región(es) donde la gráfica intersecta el eje horizontal.

Utilidad:

- Estimación inicial aproximada de la raíz.
- Identificar existencia y número de raíces en un intervalo.
- Visualizar el comportamiento de la función.

Limitación: precisión baja; depende de la escala del gráfico.

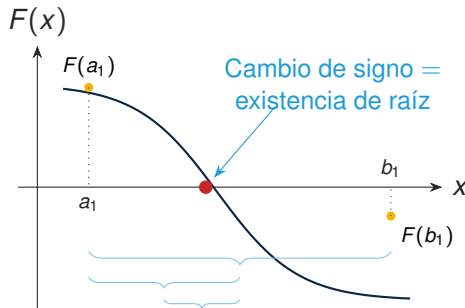


Teorema de Bolzano

Existencia

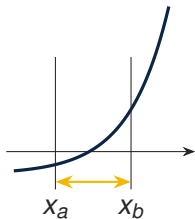
Si f es **continua** en $[a, b]$ y $f(a)f(b) < 0$, entonces existe al menos una raíz $c \in (a, b)$.

- Bernard Bolzano (1781–1848).
- El **cambio de signo** garantiza la existencia de raíz.
- Es la base teórica de los **métodos cerrados**: cada subintervalo que conserva el cambio de signo sigue encerrando la raíz.



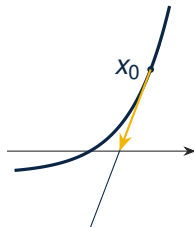
Clasificación de Métodos Iterativos

Métodos Cerrados (*Bracketing*)



- **Concepto:** requieren dos valores iniciales que *encierran* la raíz.
- **Características:** convergencia garantizada, pero generalmente lenta.
- **Ejemplos:** Bisección, Regla Falsa.

Métodos Abiertos (*Open Methods*)



- **Concepto:** requieren un valor inicial x_0 que no necesariamente encierra la raíz. Proyectan la solución.
- **Características:** convergencia rápida (cuadrática), pero riesgo de divergencia.
- **Ejemplos:** Newton–Raphson, Punto Fijo, Secante.

B. Métodos Cerrados: Bisección

Intuición geométrica: divide y vencerás

1. Iniciar con un intervalo $[a, b]$
donde los signos son opuestos.

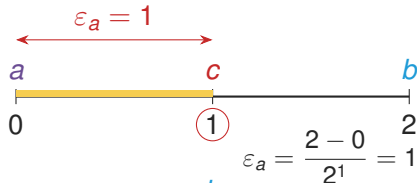
2. Calcular el punto medio

$$c = \frac{a + b}{2}$$

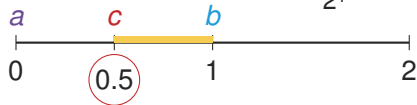
3. Desechar la mitad que **no** tiene
cambio de signo.

4. Repetir.

$i = 0$



$i = 1$



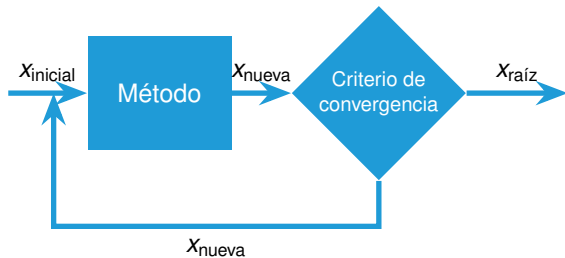
Características: convergencia **siempre garantizada** (robusto), pero **lenta**.

Algoritmo de Bisección

Pseudocódigo

```
Input:  a, b, Tol
Check:   $f(a) \cdot f(b) < 0$ 
While  $(b - a) / 2 > \text{Tol}$ :
     $c = (a + b) / 2$ 
    If  $f(c) == 0$ : Stop
    If  $f(a) \cdot f(c) < 0$ :
         $b = c$  // izquierda
    Else:
         $a = c$  // derecha
Return c
```

Flujo lógico del método



El bucle While y las condiciones If se representan por la iteración entre el *Método* y el *Criterio de convergencia*.

Análisis de Error en Bisección

Predicción *a priori*

Cota de error

En la iteración n , el error absoluto es menor que la mitad del intervalo actual:

$$|x_{\text{real}} - x_n| \leq \frac{b - a}{2^{n+1}}$$

Cálculo de iteraciones

Número de iteraciones necesarias n para una tolerancia Tol :

$$n > \frac{\ln(b - a) - \ln(Tol)}{\ln 2}$$

Ejemplo

Si $b - a = 1$ y $Tol = 10^{-4}$:

$$n > \frac{0 - (-9.21)}{0.69} \approx 13.28$$

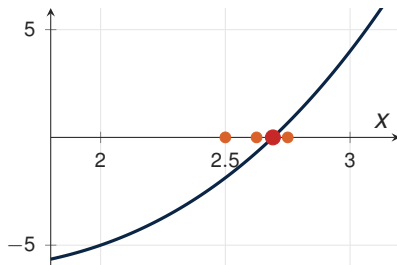
Resultado: se requieren **14 iteraciones**.

Ventaja: el número de iteraciones se conoce **antes** de ejecutar el algoritmo.

Ejemplo: Bisección para $f(x) = x^3 - 2x^2 - 5$

Intervalo inicial $[2, 3]$: $f(2) = -5 < 0$,
 $f(3) = 4 > 0$ ✓

i	a	b	c	$f(c)$
1	2.0000	3.0000	2.5000	-1.8750
2	2.5000	3.0000	2.7500	0.6719
3	2.5000	2.7500	2.6250	-0.6934
4	2.6250	2.7500	2.6875	-0.0344
5	2.6875	2.7500	2.7188	0.3127



Raíz: $x^* \approx 2.690647$

C. Métodos Abiertos

Velocidad vs. seguridad

Métodos Cerrados (Bisección)

$[-]$

Requieren un intervalo que encierre la raíz.

- ✓ Convergencia garantizada
- ✗ Convergencia lenta

vs

Métodos Abiertos (Newton, Punto Fijo)



Requieren solo un punto inicial x_0 .

- ✓ Convergencia rápida (cuando funciona)
- ✗ Riesgo de divergencia

Método del Punto Fijo

Transformación $x = g(x)$

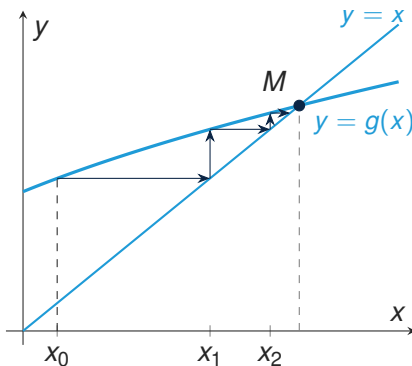
Algoritmo iterativo:

- Transformar $f(x) = 0$ en $x = g(x)$.
- Iterar:

$$x_{i+1} = g(x_i)$$

- Se busca la intersección de $y = g(x)$ con $y = x$.
- **Condición de convergencia:**
 $|g'(x)| < 1$ cerca de la raíz.

Convergencia en escalera

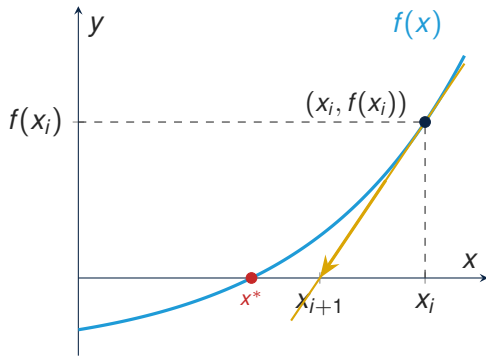


Métodos Abiertos: Newton–Raphson

Concepto: utiliza la pendiente (derivada) de la función en un punto x_i para proyectar una **recta tangente** hacia el eje x , obteniendo una mejor estimación x_{i+1} .

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

Ventaja clave: convergencia cuadrática. El error es aproximadamente proporcional al cuadrado del error anterior, *doblando* los dígitos correctos en cada iteración.



Ejemplo: Newton–Raphson para $f(x) = x^3 - 2x^2 - 5$

$$f'(x) = 3x^2 - 4x, \quad x_0 = 2$$

$$|\varepsilon_a| = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| \times 100\%$$

En solo 5 iteraciones se alcanza $|\varepsilon_a| \approx 10^{-6} \%$, frente a la lentitud de Bisección.

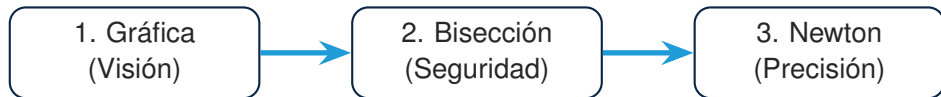
i	x_i	$f(x_i)$	$f'(x_i)$	x_{i+1}	$ \varepsilon_a (\%)$
0	2.000000	-5.0000	4.0000	3.250000	38.46
1	3.250000	8.2031	18.6875	2.811037	15.62
2	2.811037	1.4088	12.4616	2.697990	4.19
3	2.697990	0.0808	11.0455	2.690677	0.27
4	2.690677	0.0003	10.9565	2.690647	0.0011
5	2.690647	5×10^{-9}	10.9562	2.690647	2×10^{-8}

Resumen Comparativo

¿Qué herramienta usar?

Método	Tipo	Ventaja	Desventaja
Bisección	Cerrado	Garantizado (Bolzano)	Lento
Punto Fijo	Abierto	Flexible	Requiere $ g' < 1$
Newton	Abierto	Muy rápido (cuadrático)	Requiere derivada f'

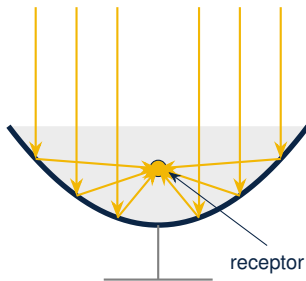
Flujo de trabajo recomendado (Ingeniería)



Aplicación en Ingeniería de Energía

Escenario: diseño y control de colectores solares parabólicos.

El desafío: para maximizar la captura de energía, se debe determinar la temperatura de operación o el ángulo de incidencia donde la eficiencia es óptima.



Contexto físico: la relación entre la radiación neta absorbida y la pérdida de calor por convección y radiación es **altamente no lineal**.

Modelado Matemático: Ecuación de Stefan–Boltzmann

$$f(T) = \varepsilon\sigma (T^4 - T_{\text{amb}}^4) - h(T - T_{\text{amb}}) = 0$$

Balance de energía: Radiación neta = Pérdida de calor + Energía almacenada

σ : Constante de Stefan–Boltzmann ($5.67 \times 10^{-8} \text{ W/m}^2\text{K}^4$)

ε : Emisividad de la superficie

h : Coeficiente de transferencia de calor por convección

T : Temperatura de la superficie (**incógnita**)

Observación: el término T^4 hace imposible despejar T explícitamente usando álgebra elemental, requiriendo métodos numéricos.

Selección del Algoritmo: ¿Por qué Newton–Raphson?

Velocidad vs. seguridad

En sistemas de control en tiempo real (ajuste de espejos solares ante nubes pasajeras), la **velocidad de cálculo es crítica**.

- Newton–Raphson ofrece una convergencia mucho más rápida que Bisección, permitiendo ajustes casi instantáneos.

Implementación

- Se calcula la derivada $f'(T)$ analíticamente una sola vez:

$$f'(T) = 4\epsilon\sigma T^3 - h$$

- **Validación del error:** la convergencia se verifica controlando el error relativo:

$$\left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| \times 100 < \text{Tolerancia}$$

Actividad Práctica: “El Copiloto Digital”

Contexto

El ingeniero moderno no solo escribe código; **supervisa y valida** soluciones generadas por herramientas avanzadas.



Consigna (20 minutos)

1. **Prompting:** utiliza una IA (ChatGPT, Claude o Gemini) para generar un script en MATLAB.
2. **Tarea:** resolver la ecuación $x^3 - 2x^2 - 5 = 0$ (o el modelo de energía) usando el método de Newton–Raphson.
3. **Restricción:** el código debe solicitar el **valor inicial** y la **tolerancia de error**.

Validación Crítica del Código

```
1  % Newton-Raphson Method Implementation
2  max_iter = 100;    % Maximum iterations
3  tol = 1e-6;        % Tolerance
4  x0 = 2;            % Initial guess
5  x = x0;
6  for i = 1:max_iter % (1)
7      f_val = x^3 - 2*x^2 - 5;
8      f_prime_val = 3*x^2 - 4*x;
9      if abs(f_prime_val) < 1e-9 % (3)
10         error('Derivative is near zero.');
```

Checklist de ingeniería:

1. **Condición de parada (1):** ¿incluye un límite máximo de iteraciones (`max_iter`) para evitar bucles infinitos?
2. **Tolerancia (2):** ¿se detiene cuando el error es menor que `tol`?
3. **Singularidades (3):** ¿maneja el caso $f'(x) \approx 0$? (riesgo de división por cero).

Lección: La IA genera la sintaxis; el ingeniero **garantiza** la lógica numérica.

Referencias Bibliográficas

1. Chapra, S. C., & Canale, R. P. *Métodos Numéricos para Ingenieros*. McGraw-Hill Education.
2. Burden, R. L., & Faires, J. D. *Análisis Numérico*. Cengage Learning.
3. Nakamura, S. *Métodos Numéricos Aplicados con Software*. Prentice Hall.
4. Kiusalaas, J. *Numerical Methods in Engineering with MATLAB*. Cambridge University Press.